

Why This Document

This compares **Qdrant** (a standalone, purpose-built vector database) against **pgvector** (a vector-search extension added directly to PostgreSQL) — the two realistic options for storing and searching the embeddings this project uses for retrieval, signal matching, and the safety interlock.

This project currently uses pgvector. Below is an honest, side-by-side comparison, followed by our reasoning for that choice. The final call on whether to keep it that way is yours.

Why Cloud-Only Options Like Pinecone Weren't Shortlisted

Pinecone-style cloud-only vector databases do not meet DPDP compliance requirements for this project's sensitive personal and health-related data. They only exist as someone else's hosted service, with no option to run them inside our own infrastructure — every embedding generated from real conversations and clinical content would be sent to and stored on that company's own servers, in regions we can't choose or verify, as a matter of how the product works, not a configurable choice.

In short, that means:

- **No self-hosting option** — sensitive data must leave to a third party's servers, with no way to keep it inside infrastructure this project controls.
- **No guaranteed data residency** — the data is stored wherever that company operates, not necessarily anywhere DPDP-compliant.
- **A new, unverified third party gains access to sensitive data**, without the ability to fully audit or control how it's handled.

Qdrant avoids this: it's open-source and can be self-hosted inside infrastructure we already control, keeping data fully within compliant boundaries — which is why it gets a fair comparison below. pgvector goes further still — it isn't a new vendor at all, just a capability added to the database this project already runs.

Side-by-Side Comparison

	Qdrant	pgvector (PostgreSQL)
What it is	A separate, purpose-built database made only for vector search	An extension that adds vector search directly into a regular PostgreSQL database
Infrastructure	A second service to deploy, monitor, back up, and secure — in addition to your main database	No second system — vectors live in the same database already running everything else

	Qdrant	pgvector (PostgreSQL)
Combining a vector match with real data	Two steps: search Qdrant for a match, then separately look up the full record in Postgres, then combine them in application code	One single query can find the nearest match and fetch the full record (e.g. the exact clinical response to send) together
Data consistency	Two separate systems can fall out of sync if one update succeeds and the other fails	Vector data and regular data are updated together, in the same transaction — always consistent
Performance at very large scale (millions–billions of vectors)	Built specifically for this — generally the faster option at extreme scale	Performs well up to tens of millions of vectors with proper indexing; not built to out-perform a dedicated engine at extreme scale
This project's actual scale today	—	About 8,500 knowledge-base records total, ~8,300 of them vectorized — comfortably within pgvector's efficient range
Compliance / data security surface	A second system that needs its own access controls, audit trail, and security review	One system already being secured and audited — a smaller surface to review and certify
Operational cost	Additional hosting and maintenance cost for a separate service	No added infrastructure — reuses what's already provisioned
Team familiarity	Requires learning and operating a new, specialized system	The team already runs and knows PostgreSQL day to day
Specialized vector features	More advanced tuning options, multi-vector support, filtering optimizations built specifically for vector workloads	Covers everything this project currently needs; fewer specialized knobs than a dedicated engine

Both are genuinely good, actively maintained technologies — this is a question of fit for this specific project, not one being objectively “better” than the other in general.

Why We Recommend pgvector For This Project

- **Every record that has an embedding also carries the real data needed the moment a match is found** — the clinical meaning, the exact response script, the escalation target. pgvector fetches the match and that real data together in one query. With a separate vector database, that would take two round-trips and extra code to keep both systems in sync — more moving parts, more ways for them to drift apart.

- **The current data size doesn't need a specialized engine.** At roughly 8,500 records, pgvector has no meaningful performance disadvantage — the scale where a dedicated vector database clearly pulls ahead is millions of records and beyond.
- **Fewer systems to secure and maintain.** For a mental-health product handling sensitive personal data, one database to audit and secure is a smaller, simpler compliance surface than two.
- **No added infrastructure cost or new operational skill required** — the team already runs PostgreSQL; adding pgvector was a natural extension, not a new system to learn.
- **This isn't a permanent, unchangeable decision.** If the data volume grows dramatically, or pure vector-search speed becomes an actual bottleneck later, Qdrant remains a legitimate option to revisit at that point — this is the right fit for today's real scale and architecture, not a one-way door.

This is our recommendation based on the project's current scale and architecture — the final decision is yours.